



프로그램 언어

9주차



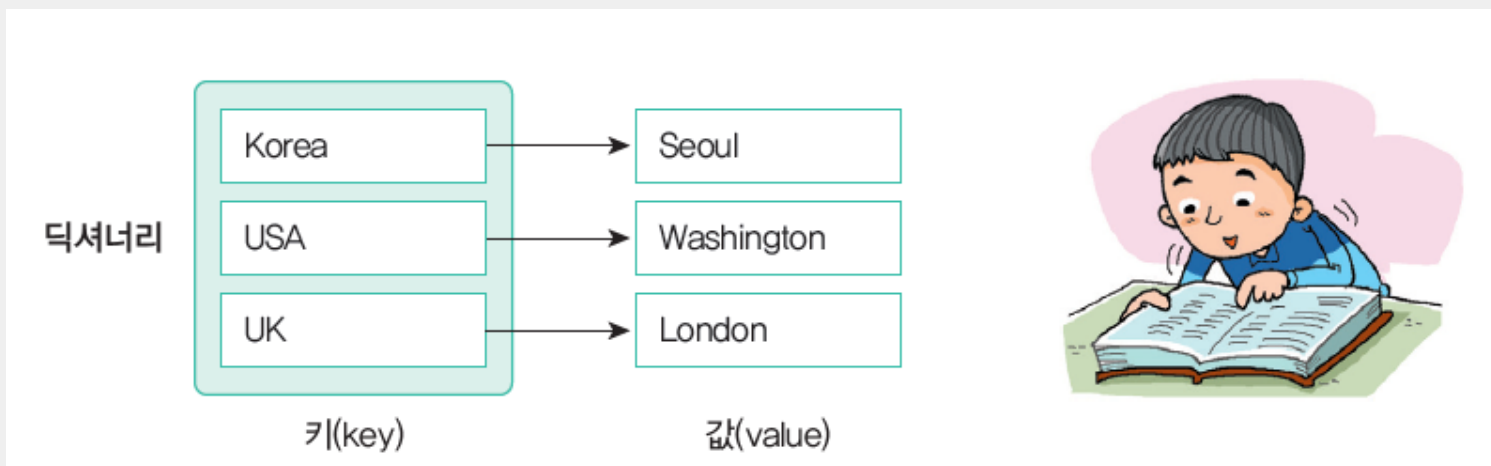
인하대학교



딕셔너리 (Dictionary)

딕셔너리

- 딕셔너리(dictionary)도 값을 저장하는 자료구조. 하지만 딕셔너리에는 값(value)과 관련된 키(key)도 저장된다. 키값은 중복되지 않는 값



딕셔너리 생성

Syntax: 딕셔너리

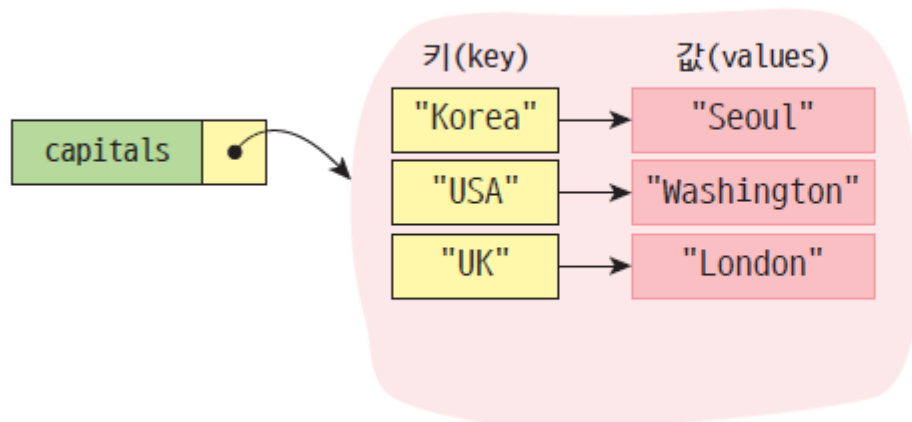
형식 딕셔너리_이름 = { 키1:값1, 키2:값2, 키3:값3, ... }

예 capitals = { } # ①
 capitals = { "Korea": "Seoul" , "USA": "Washington", "UK": "London" } # ②

공백 딕셔너리를 생성한다.

키

값



딕셔너리는 키와
값으로 이루어집니다.



딕셔너리 생성

- 중괄호{ }로 묶여 있으며 **키**와 **값**의 쌍으로 이루어짐
- key, value, item

딕셔너리변수 = { 키1:값1 , 키2:값2, 키3:값3, ... }

```
dic1={ 1:'a', 2:'b', 3:'c'}  
dic1
```

출력 결과

```
{1: 'a', 2: 'b', 3: 'c'}
```

```
dic2={ 'a':1, 'b':2, 'c':3}  
dic2
```

출력 결과

```
{'b': 2, 'c': 3, 'a': 1}
```

항목 탐색하기

```
>>> capitals = {"Korea": "Seoul", "USA": "Washington", "UK": "London"}
```

```
>>> print( capitals["Korea"] )
```

```
Seoul
```

```
>>> print( capitals["France"] )
```

```
...
```

```
KeyError: "France"
```

```
>>> print( capitals.get("France", "해당 키가 없습니다." ) )
```

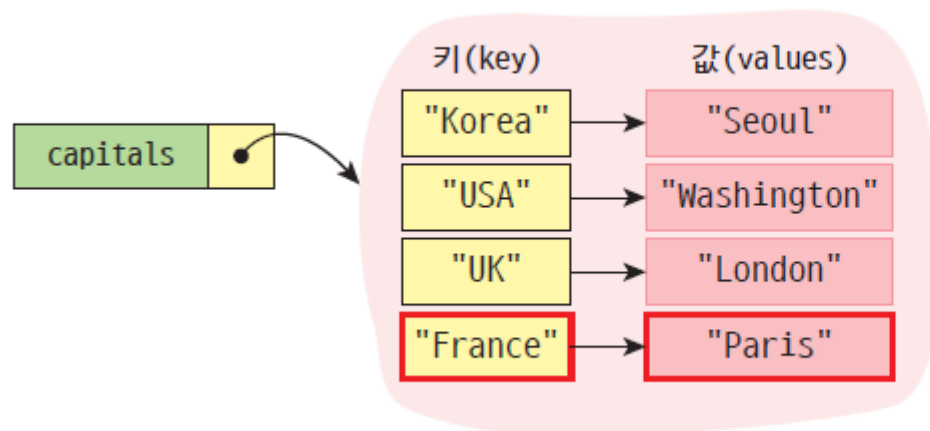
```
해당 키가 없습니다.
```

프로그램이 오류로 중단되지 않게 하려면 이렇게 해야 함!

딕셔너리 생성(항목 추가하기)

```
capitals = {}  
capitals["Korea"] = "Seoul"  
capitals["USA"] = "Washington"  
capitals["UK"] = "London"  
capitals["France"] = "Paris"  
print(capitals)
```

```
{'Korea': 'Seoul', 'USA': 'Washington', 'UK': 'London', 'France': 'Paris'}
```



딕셔너리에 추가할 때는
[] 연산자를 사용하세요.



함수

- '딕셔너리이름.get(키)' 함수 : 키가 없을 때 아무것도 반환하지 않음

```
student1.get('이름')
```

출력 결과

```
'홍길동'
```

키	값
학번	1000
이름	홍길동
학과	열공학과

- '딕셔너리이름.keys()' 함수 - 딕셔너리의 모든 키 반환

```
student1.keys()
```

출력 결과

```
dict_keys(['학번', '이름', '학과', '연락처'])
```

- 'list(딕셔너리이름.keys())' 함수 - 앞에 dict_keys 를 빼줌

```
list(student1.keys())
```

출력 결과

```
['학번', '이름', '학과', '연락처']
```


함수

- '딕셔너리이름.values()' 함수 - 딕셔너리의 모든 값 반환

```
student1.values()
```

출력 결과

```
dict_values([1000, '홍길동', '파이썬학과', '010-1111-2222'])
```

- '딕셔너리이름.items()' 함수

```
student1.items()
```

출력 결과

```
dict_items([('학번', 1000), ('이름', '홍길동'), ('학과', '파이썬학과'), ('연락처', '010-1111-2222')])
```

- In - 딕셔너리에 키가 있으면 True를, 없으면 False 반환

```
'이름' in student1  
'주소' in student1
```

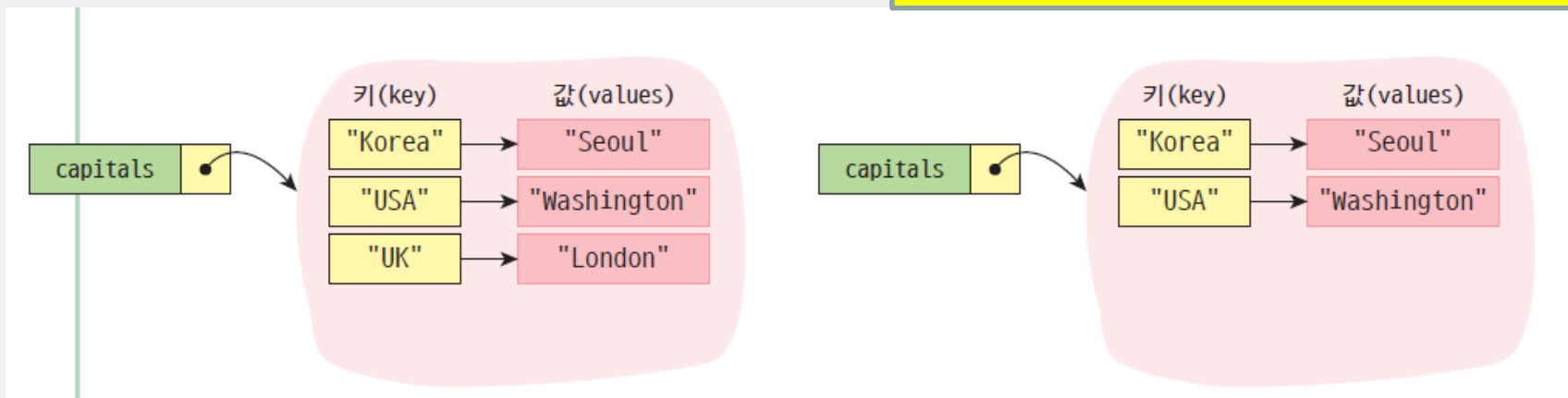
출력 결과

```
True  
False
```

항목 삭제하기

```
city = capitals.pop("UK")
```

만약 주어진 키를 가진 항목이 없으면 Error 발생



```
if "UK" in capitals :  
    capitals.pop("UK")
```

```
del(capital['UK'])
```

항목 방문하기

```
capitals = {"Korea": "Seoul", "USA": "Washington", "UK": "London"}  
for i in capitals :  
    print( i, ":", capitals[i])
```

```
Korea : Seoul  
USA : Washington  
UK : London
```

```
capitals = {"Korea": "Seoul", "USA": "Washington", "UK": "London"}  
for key, value in capitals.items():  
    print( key, ":", value )
```

```
Korea : Seoul  
USA : Washington  
UK : London
```



기타 연산

```
capitals = {"Korea": "Seoul", "USA": "Washington", "UK": "London"}  
print( capitals.keys())  
print( capitals.values())
```

```
dict_keys(['Korea', 'USA', 'UK'])  
dict_values(['Seoul', 'Washington', 'London'])
```

```
for key in sorted( capitals.keys()):  
    print(key, end=" ")
```

```
Korea UK USA
```

딕셔너리 메소드

연산	설명
<code>d = dict()</code>	공백 딕셔너리를 생성한다.
<code>d={k₁:v₁,k₂:v₂, ...,k_n:v_n}</code>	초기값으로 딕셔너리를 생성한다.
<code>len(d)</code>	딕셔너리에 저장된 항목의 개수를 반환한다.
<code>k in d</code>	k가 딕셔너리 d 안에 있는지 여부를 반환한다.
<code>k not in d</code>	k가 딕셔너리 d 안에 없으면 True를 반환한다.
<code>d[key] = value</code>	딕셔너리에 키와 값을 저장한다.
<code>v = d[key]</code>	딕셔너리에서 key에 해당되는 값을 반환한다.
<code>d.get(key, default)</code>	주어진 키를 가지고 값을 찾는다. 만약 없으면 default 값이 반환된다.
<code>d.pop(key)</code>	항목을 삭제한다.
<code>d.values()</code>	딕셔너리 안의 모든 값의 시퀀스를 반환한다.
<code>d.keys()</code>	딕셔너리 안의 모든 키의 시퀀스를 반환한다.
<code>d.items()</code>	딕셔너리 안의 모든 (키, 값)을 반환한다.

실습(1/6)

- for문을 활용하여 딕셔너리의 모든 값 출력

```
이름 --> 아이오아이  
구성원수 --> 11  
대표곡 --> 픽미픽미픽미업  
데뷔 --> 프로듀스101
```

```
1 singer={}  
2  
3 singer['이름']='아이오아이'  
4 singer['구성원수']=11  
5 singer['데뷔']='프로듀스101'  
6 singer['대표곡']='픽미픽미픽미업'  
7  
8 for k in singer.keys():  
9     print(k, singer[k])
```

실습(2/6): 영한 사전

단어를 입력하시오: one
하나

단어를 입력하시오: two
둘



코딩:

```
english_dict = {}                                # 공백 딕셔너리를 생성한다.

english_dict["one"]="하나"                        # 딕셔너리에 단어와 의미를 추가한다.
english_dict["two"]="둘"
english_dict["three"]="셋"

word =input("단어를 입력하시오: ")
print (english_dict[word])
```


실습3/6

- 디렉터리를 이용하여 이름을 입력하면 학과가 출력되도록 프로그래밍

이름을 입력하시오: **홍길동**

미디어커뮤니케이션학과

이름을 입력하시오: **홍길순**

경제학과

이름을 입력하시오: **끝**



```
students = {}
```

```
students['홍길동'] = '미디어커뮤니케이션학과'
```

```
students['홍길순'] = '경제학과'
```

```
students['심청'] = '국제통상학과'
```

```
while True:
```

```
    name = input(" 이름을 입력하시오: ")
```

```
    if name=='끝':
```

```
        break
```

```
    print(students[name])
```



실습4/6: 학생 성적 처리

```
score_dic = {  
    "Kim": [99, 83, 95],  
    "Lee": [68, 45, 78],  
    "Choi": [25, 56, 69]  
}
```

Kim 의 평균성적= 92.33333333333333
Lee 의 평균성적= 63.666666666666664
Choi 의 평균성적= 50.0

코딩:

```
score_dic = {  
    "Kim": [99, 83, 95],  
    "Lee": [68, 45, 78],  
    "Choi": [25, 56, 69]  
}
```

```
for name, scores in score_dic.items():  
    print(name, "의 평균성적=", sum(scores)/len(scores))
```

실습5/6 : 편의점 재고

- 편의점에서 재고 관리를 수행하는 프로그램을 작성해보자. 편의점에서 판매하는 물건의 재고를 딕셔너리에 저장한다.

물건의 이름을 입력하시오: **콜라**

4



커피음료: 7

펜: 3

종이컵: 2

우유: 1

콜라 4

책: 5

```
menus = { "커피음료": 7, "펜": 3, "종이컵": 2, "우유": 1, "콜라": 4, "책": 5 }
```

```
item = input("물건의 이름을 입력하시오: ")
```

```
print (menus[item])
```



도전문제

위의 프로그램을 편의점의 재고를 관리하는 프로그램으로 업그레이드해보자. 즉 재고를 증가, 또는 감소시킬 수도 있도록 코드를 추가해보자. 간단한 메뉴도 만들어보자.

실습6/6: 연락처 관리

1. 연락처 추가
2. 연락처 삭제
3. 연락처 검색
4. 연락처 출력
5. 종료

메뉴 항목을 선택하시오: 1

이름: KIM

전화번호: 123-4567

1. 연락처 추가
2. 연락처 삭제
3. 연락처 검색
4. 연락처 출력
5. 종료

메뉴 항목을 선택하시오: 4

KIM 의 전화번호: 123-4567

...



```
address_book = {}
```

```
# 공백 딕셔너리를 생성한다.
```

```
while True :
```

```
    print("1. 연락처 추가")
```

```
    print("2. 연락처 삭제")
```

```
    print("3. 연락처 검색")
```

```
    print("4. 연락처 출력")
```

```
    print("5. 종료")
```

```
    user = int(input("메뉴 항목을 선택하시오: "))
```

```
    if user ==1 :
```

```
        name =input("이름: ")
```

```
        number =input("전화번호:")
```

```
        address_book[name]= number
```

```
# name과 number를 추가한다.
```

```
    elif user ==2 :
```

```
        name =input("이름: ")
```

```
        address_book.pop(name)
```

```
# name을 키로 가지고 항목을 삭제한다.
```

```
    elif user ==3 :
```

```
        name =input("이름: ")
```

```
        print(name, "의 연락처 : ",address_book[name])
```

```
    elif user ==4 :
```

```
        for key in sorted(address_book):
```

```
            print(key,"의 전화번호:", address_book[key])
```

```
    else :
```

```
        break
```




문자열(string)

문자열과 리스트

- 문자열은 시퀀스(순서가 있는 요소로 이루어진 자료형)
- 문자들이 모인 리스트라고 생각할 수 있다.
- 리스트의 indexing, slicing 사용가능

						[6:10]					
0	1	2	3	4	5	6	7	8	9	10	11
M	o	n	t	y		P	y	t	h	o	n
-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
[-12:-7]											



```
s = "Monty Python"
print(s[0])           # M
print(s[6:10])        # Pyth
print(s[-12:-7])      # Monty
```

실습

- 입력한 글자수 확인

```
sosok=input('소속을 입력하시오')  
print(sosok, '은', len(sosok), '글자입니다')
```

‘소속을 입력하시오’ **인하대학교**
인하대학교는 5 글자입니다

문자열 슬라이싱



인하대학교

```
univ = "인하대학교"
```

```
for i in univ:  
    print(i)
```

```
>>> s = "Monty Python"  
>>> s[:2]  
"Mo"  
>>> s[4:]  
"y Python"
```

```
>>> s = "Monty Python"  
>>> s[:2] + s[2:]  
"Monty Python"  
>>> s[:4] + s[4:]  
"Monty Python"
```

```
>>> s = "Monty Python"  
>>> s[:]  
"Monty Python"
```

문자열 슬라이싱

```
>>> message="see you at noon"
>>> low = message[:5]
>>> high = message[5:]
>>> low
"see y"
>>> high
"ou at noon"
```

```
>>> reg= "980326"
```

```
>>> print(reg[0:2]+"년")
98년
>>> print(reg[2:4]+"월")
03월
>>> print(reg[4:6]+"일")
26일
```

실습(1/2)

주민등록번호 13
여자를 의미한다
여자)을 출력하는

```
jumin=input("주민번호 13자리 입력:")  
if jumin[6]=='1' or jumin[6]=='3' :  
    print("남자")  
elif jumin[6]=='2' or jumin[6]=='4' :  
    print("여자")  
else:  
    print("잘못 입력하셨습니다")
```

2, 4는
후 성별 (남자,

실행화면

```
>> 주민번호 13자리  
>> 남자
```

8210102635210

```
jumin=input("주민번호 13자리 입력:")  
if jumin[6]=='1' or jumin[6]=='3' :  
    print("남자")  
else:  
    print("여자")
```

실습(2/2) : 슬라이싱

문자열 입력 : Hello

Hello 0:4

ello 1:4

llo 2:4

lo 3:4

o 4:4

H 0:1

He 0:2

Hel 0:3

Hell 0:4

Hello 0:5

```
ch=input('문자열입력')
```

```
for i in range(len(ch)):
    print(ch[i:])
```

```
for i in range(len(ch)):
    print(ch[:i+1])
```

문자열은 불변 객체: 변경 불가능

```
>>> word = "abcdef"
>>> word[0]="A "
...
TypeError: "str" object does not support item assignment
```

```
>>> word = "abcdef"
>>> word = "A" + word[1:]
>>> word
"Aabcdef"

>>> word = "abcdef"
>>> id(word)
2387542507952
>>> word = "A" + word[1:]
>>> id(word)
2387536819184
```

- 문자열 변경 : `replace('기존 문자열', '새 문자열')`

```
ss = '열심히 파이썬을 공부중~~'
ss.replace('파이썬', 'Python')
```

출력 결과

```
'열심히 Python을 공부중~~'
```


문자열 비교

- ==, !=, <, > 연산자를 문자열에 적용할 수 있다.

```
a = input("문자열을 입력하시오: ")
b = input("문자열을 입력하시오: ")
if( a < b ):
    print(a, "이(가) 앞에 있음")
else:
    print(b, "이(가) 앞에 있음")
```

```
문자열을 입력하시오: apple
문자열을 입력하시오: orange
apple 이(가) 앞에 있음
```

실습: 회문 검사하기

- 회문(palindrome)은 앞으로 읽으나 뒤로 읽으나 동일한 문장이다. 예를 들어서 " mom ", " civic ", " dad " 등이 회문의 예이다. 사용자로부터 문자열을 입력받고 회문인지를 검사하는 프로그램을 작성하여 보자.

문자열을 입력하시오: dad
회문입니다.

리스트[::-1]

```
s = input("문자열을 입력하시오: ")
```

```
s1 = s[::-1] # 문자열을 거꾸로 만든다.
```

```
if( s == s1 ):  
    print("회문입니다.")
```

```
else:  
    print("회문이 아닙니다.")
```

문자열 메소드

- upper() 함수 : 소문자를 대문자로 변경
- lower() 함수 : 대문자를 소문자로 변경
- swapcase() 함수 : 대소문자를 상호 변환
- title() 함수 : 각 단어의 제일 앞 글자만 대문자로 변환

```
ss = 'Python is Easy. 그래서 programming이 재미있지 말입니다 ^^'  
ss.upper()  
ss.lower()  
ss.swapcase()  
ss.title()
```

출력 결과

```
'PYTHON IS EASY. 그래서 PROGRAMMING이 재미있지 말입니다 ^^'  
'python is easy. 그래서 programming이 재미있지 말입니다 ^^'  
'pYTHON IS eASY. 그래서 PROGRAMMING이 재미있지 말입니다 ^^'  
'Python Is Easy. 그래서 Programming이 재미있지 말입니다 ^^'
```



문자열 메소드

```
>>> s = "i am a student."
```

```
>>> s.capitalize()
```

```
"I am a student."
```

```
>>> s.title()
```

```
"I Am A Student."
```

문자열 메소드

❖ 문자열 공백 제거 : strip(), rstrip(), lstrip()

단 문자열中间的 공백은 제거되지 않음

```
ss = '   파   이   션   '
ss.strip()
ss.rstrip()
ss.lstrip()
```

출력 결과

```
'파   이   션'
'   파   이   션'
'파   이   션   '
```

```
ss = '----파---이---션----'
print(ss.strip('-'))
ss = '<<<파 << 이 >> 션>>>'
print(ss.strip('<>'))
```

출력 결과

```
파---이---션
파 << 이 >> 션
```

문자열 메소드: 공백 제거

```
>>> s = " Hello, World! "  
>>> s.strip()  
"Hello, World!"
```

```
>>> s = "#####this is example#####"
>>> s.strip("#")
"this is example"
```

```
>>> s = "#####this is example#####"
>>> s.lstrip("#")
"this is example#####"
>>> s.rstrip("#")
"#####this is example"
```

문자열 메소드

- center(숫자,채움문자) : 숫자만큼 전체 자릿수를 잡은 후 문자열을 가운데 배치하고 공백부분을 채움문자로 채움
- ljust() : 왼쪽에 붙여 출력
- rjust() : 오른쪽에 붙여 출력
- zfill() : 오른쪽으로 붙여 쓰고 왼쪽 빈 공간은 0으로 채움

```
ss = '파이썬'  
ss.center(10)  
ss.center(10, '-')  
ss.ljust(10)  
ss.rjust(10)  
ss.zfill(10)
```

출력 결과

```
'   파이썬   '  
'---파이썬---'  
'파이썬      '  
'          파이썬'  
'0000000파이썬'
```


문자열 메소드: 찾기 및 바꾸기

#find(찾을 문자, 찾기 시작할 위치, 찾기를 끝맺을 위치)

문자열의 왼쪽부터 문자를 찾음.

찾으면 처음 찾은 문자의 위치를 반환. 못찾으면 '-1'을 반환

단, 찾기 시작할 위치와 찾기를 끝맺을 위치는 별도 지정 없으면 문자열 전체를 탐색함.

#rfind(찾을 문자, 찾기 시작할 위치, 찾기를 끝맺을 위치)

문자열에서 중복된 문자가 있으면, 문자열 중 가장 끝에 있는 위치를 반환함

```
>>> s = "www.naver.co.kr"
```

```
>>> s.find(".kr")
```

```
12
```

```
>>> s = "Let it be, let it be, let it be"
```

```
>>> s.rfind("let")
```

```
22
```

```
>>> s = "www.naver.co.kr"
```

```
>>> s.count(".")
```

```
3
```

문자열 메소드: 문자열 분해하기

- `split()` : 구분자를 이용하여 문자열을 단어로 구분하여 리스트로 만드는 함수

```
>>> s = "Welcome to Python"
```

```
>>> s.split()
```

```
["Welcome", "to", "Python"]
```

```
>>> s = "Hello, World!"
```

```
>>> s.split(",")
```

```
["Hello", " World!"]
```

```
>>> list("Hello, World!")
```

```
["H", "e", "l", "l", "o", ",", " ", "W", "o", "r", "l", "d", "!"]
```

문자열 메소드: 문자열 결합하기

- join() :단어들을 모아서 **하나의 문자열로** 만드는 함수

```
>>> ",".join(["apple", "grape", "banana"])  
"apple,grape,banana"
```

```
#010.1234.5678
```

```
>>> "-".join("010.1234.5678".split("."))  
"010-1234-5678"
```

```
>>>a= "Hello, World!"
```

```
>>> list(a)
```

```
["H", "e", "l", "l", "o", ",", " ", "W", "o", "r", "l", "d", "!"]
```

```
>>>''.join(a)
```

```
'Hello, World!'
```

실습: 문자열의 공통 문자

- 중복되지 않은 단어의 개수 세기

입력 텍스트: I have a dream that one day every valley shall be exalted and every hill and mountain shall be made low

사용된 단어의 개수 = 17

{"be", "and", "shall", "low", "have", "made", "one", "exalted", "every", "mountain", "I", "that", "valley", "hill", "day", "a", "dream"}

```
txt = input("입력 텍스트: ")  
words = txt.split()  
unique = set(words)      # 집합으로 만들면 자동적으로 중복을 제거한다.  
  
print("사용된 단어의 개수= ", len(unique))  
print(unique)
```

문자열 메소드: 시작문자, 끝문자

startswith(시작하는 문자(열), 시작지점)

문자열이 특정문자로 시작하는지 알려줌. **True**나 **False**를 반환.

시작지점을 별도 지정하지 않으면 처음부터 확인함.

endswith(끝나는 문자(열), 문자열의 시작, 문자열의 끝)

문자열이 특정문자로 끝나는지 알려줌.

```
s = input("파이썬 소스 파일 이름을 입력하시오: ")

if s.endswith(".py"):
    print("올바른 파일 이름입니다")
else :
    print("올바른 파일 이름이 아닙니다.")
```

```
파이썬 소스 파일 이름을 입력하시오: aaa.py
올바른 파일 이름입니다
```



문자열 메소드: 문자 분류

```
>>> "ABCAbc".isalpha()
```

```
True
```

```
>>> "abc".isalpha()
```

```
True
```

```
>>> "123".isdigit()
```

```
True
```

```
>>> "abc".islower()
```

```
True
```

실습 1/5: 문자열 분석

- 문자열 안에 있는 문자의 개수, 숫자의 개수, 공백의 개수를 계산하는 프로그램을 작성하여 보자.

문자열을 입력하시오: A picture is worth a thousand words.

```
{"digits": 0, "spaces": 6, "alphas": 29}
```



```
sentence = input("문자열을 입력하시오: ")

table = {"alphas":0,"digits":0,"spaces":0 }

for i in sentence:
    if i.isalpha():
        table["alphas"]+=1
    if i.isdigit():
        table["digits"]+=1
    if i.isspace():
        table["spaces"]+=1

print(table)
```



실습2/5: 머리 글자어 만들기

- 머리 글자어(acronym)은 NATO(North Atlantic Treaty Organization)처럼 각 단어의 첫 글자를 모아서 만든 문자열이다. 사용자가 문장을 입력하면 해당되는 머리 글자어를 출력하는 프로그램을 작성하여 보자.

문자열을 입력하시오: North Atlantic Treaty Organization
NATO

```
phrase = input("문자열을 입력하시오: ")

acronym = ""

# 대문자로 만든 후에 단어들로 분리한다.
for word in phrase.upper().split():
    acronym += word[0]           # 단어를 첫 글자만을 acronym에 추가한다.

print( acronym )
```



실습3/5: 이메일 주소 분석

- 이메일 주소에서 아이디와 도메인을 구분하는 프로그램을 작성하여 보자.

이메일 주소를 입력하시오: aaa@google.com

aaa@google.com

아이디:aaa

도메인:google.com

코딩:



인하대학교

```
address=input("이메일 주소를 입력하시오: ")
add2 = address.split("@")
id=add2[0]
domain=add2[1]
print(address)
print("아이디:", id)
print("도메인:", domain)
```

실습4/5: 메시지 처리

- 단어의 개수 추출

```
t = "Python is very easy and powerful!"
```

```
length = len(t.split())  
print(length)
```

```
6
```

실습5/5: 단어 카운터 만들기



인하대학교

```
text_data = "Create the highest, grandest vision possible for your life, because you become what you believe"
```

```
word_dic = {}
for w in text_data.split():
    if w in word_dic:
        word_dic[w] += 1
    else:
        word_dic[w] = 1

for w, count in sorted(word_dic.items()):
    print(w, "의 등장횟수=", count)
```

단어들과 출현 횟수를 저장하는 딕셔너리를 생성
텍스트를 단어들로 분리하여 반복한다.
단어가 이미 딕셔너리에 있으면
출현 횟수를 1 증가한다.
처음 나온 단어이면 1로 초기화한다.

키와 값을 정렬하여 반복 처리한다.

```
Create 의 등장횟수= 1
because 의 등장횟수= 1
become 의 등장횟수= 1
believe 의 등장횟수= 1
...
```